Singleton Pattern

- 1 概述
- 2 Lazy Singleton
  - 2.1 基础版
  - 2.2 升级版
  - 2.3 进阶版
- 3 Eager Singleton
- 4 Meyers Singleton
- 5 总结
- 6 参考文献

1 概述

单例模式(Singleton Pattern，也称为单件模式)，是为了保证一个类仅有一个实例，并提供一个访问它的全局访问点，该实例被所有程序模块共享。

- 定义一个单例类：
  1. 私有化它的构造函数和析构函数，防止外界创建单例类的对象；
  2. 私有化或者删除它的拷贝构造函数；
  3. 使用类的私有静态指针变量指向类的唯一实例；
  4. 使用一个公有的静态方法获取该实例。
- 使用场景：
  1. 要求生产唯一序列号；
  2. WEB 中的计数器，不用每次刷新都在数据库里加一次，用单例先缓存起来；
  3. 创建对象需要消耗的资源过多，比如 I/O 与数据库的连接等。
- 优点：
  1. 在内存中只有一个实例，减少了内存的开销，尤其是在频繁的创建和销毁实例的场景下；
  2. 避免对资源的多重占用。
- 缺点
  1. 不能继承；
  2. 与单一原则冲突（一个类应该之关心内部逻辑，而不关系外面怎么实例化）。

在 C++11 之前需要考虑多线程情况下的安全问题，可通过同步锁解决，防止多线程同时进入造成多次实例化。

2 Lazy Singleton

单例实例在第一次使用时才进行初始化，如果没有地方用，单例也就不会实例话，相当于没用。

2.1 基础版

```
class Singleton
{
private:
    static Singleton *instance;

private:
    Singleton() {}
    Singleton(const Singleton &) = delete;
    ~Singleton() {}

public:
    static Singleton *getInstance()
    {
        if (instance == nullptr)
        {
            instance = new Singleton();
        }
        return instance;
    }
};
```

这种方式理解简单，所以也被称为教学版，但是存在线程不安全的问题。当多个线程同时使用单例时，可能存在第一次初始化几个实例的问题。

2.2 升级版

- 使用智能指针

```
class Singleton
{
private:
    static std::unique_ptr<Singleton> singleton;

private:
    Singleton() {}
    Singleton(const Singleton &) = delete;
    ~Singleton() {}

public:
    static std::unique_ptr<Singleton> &getInstance()
    {
        if (singleton == nullptr)
        {
            singleton = std::make_unique<Singleton>(new Singleton);
        }
        return singleton;
    }
};
```

- 使用静态的嵌套类对象

```
class Singleton
{
private:
    static Singleton *instance;

private:
    class Deletor
    {
    public:
        ~Deletor()
        {
            if (Singleton::instance != nullptr)
            {
                delete Singleton::instance;
            }
        }
    };
    static Deletor deletor;

private:
    Singleton() {}
    Singleton(const Singleton &) = delete;
    ~Singleton() {}

public:
    static Singleton *getInstance()
    {
        if (instance == nullptr)
        {
            instance = new Singleton();
        }
        return instance;
    }
};
```

初始化时，在单例类内定义私有的专门用于释放的静态成员 `deletor` ，当程序运行结束时，利用程序在结束时析构全局变量的特性，系统会调用静态成员 `deletor` 的析构函数，该析构函数会删除单例的唯一实例，解决了内存泄漏的问题。

这个方式在单线程环境下是正确的，但是拿到多线程环境下就会出现 race condition，要使其能在多线程环环境下正常，可以考虑加锁。

2.3 进阶版

```
class Singleton
{
private:
    static Singleton *instance;

private:
    Singleton() {}
    Singleton(const Singleton &);
    ~Singleton() {}

private:
    class Deletor
    {
    public:
        ~Deletor()
        {
            if (Singleton::instance != nullptr)
            {
                delete Singleton::instance;
            }
        }
    };
    static Deletor deletor;

public:
    static Singleton *getInstance()
    {
        if (instance == nullptr)
        {
            Lock lock; // 基于作用域的加锁，超出作用域，自动调用析构函数解锁
            if (instance == nullptr)
            {
                instance = new Singleton();
            }
        }
        return instance;
    }
};
```

线程安全问题仅出现在第一次初始化的时候，后面获取该实例的时候并不会遇到，也就没有必要再使用 lock，因为每次获取锁的状态都是有性能损耗的。

双检测锁很好地解决了这个问题，它通过加锁前检测是否已经初始化，避免了每次获取实例时都要首先获取锁资源。

加入 DCL: Double-Checked Locking Pattern 后，其实还是有问题的，关于 memory model。

在某些内存模型中或者是由于编译器的优化以及运行时优化等原因，使得 instance 虽然已经不是 nullptr 但是其所指对象还没有完成构造，这种情况下，另一个线程如果调用 `getInstance()` 就有可能使用到一个不完全初始化的对象。换句话说，就是代码中： `if(instance == NULL)` 和 `instance = new Singleton();` 没有正确的同步，在某种情况下会出现 `new` 返回了地址赋值给 `instance` 变量而 `Singleton` 此时还没有构造完全，当另一个线程随后运行到 `if(instance == NULL)` 时将不会进入，从而返回了不完全的实例对象给用户使用，造成了严重的错误。在 C++11 没有出来的时候，只能靠插入两个 memory barrier 来解决这个错误，但是 C++11 引进了 memory model，提供了 Atomic 实现内存的同步访问，即不同线程总是获取对象修改前或修改后的值，无法在对象修改期间获得该对象。

因此，在有了 C++11 后就可以正确的跨平台的实现 DCL 模式了，利用 atomic，代码如下：

```
atomic<Widget *> Widget::pInstance{nullptr};
Widget *Widget::Instance()
{
    if (pInstance == nullptr)
    {
        lock_guard<mutex> lock{mutW};
        if (pInstance == nullptr)
        {
            pInstance = new Widget();
        }
    }
    return pInstance;
}
```

C++11 中的 atomic 类的默认 `memory_order_seq_cst` 保证了 5、10 行代码的正确同步，由于上面的 atomic 需要一些性能上的损失，因此我们可以写一个优化的版本：

```
atomic<Widget *> Widget::pInstance{nullptr};
Widget *Widget::Instance()
{
    Widget *p = pInstance;
    if (p == nullptr)
    {
        lock_guard<mutex> lock{mutW};
        if ((p = pInstance) == nullptr)
        {
            pInstance = p = new Widget();
        }
    }
    return p;
}
```

3 Eager Singleton

单例实例在程序运行时被立即执行初始化。

```
class Singleton
{
private:
    static Singleton instance;

private:
    Singleton() {}
    ~Singleton() {}
    Singleton(const Singleton &);
    Singleton &operator=(const Singleton &);

public:
    static Singleton &getInstance()
    {
        return instance;
    }
};
```

由于在 main 函数之前初始化，所以没有线程安全的问题。但是潜在问题在于 no-local static 对象（函数外的 static 对象）在不同编译单元中的初始化顺序是未定义的。也即 `static Singleton instance;` 和 `static Singleton& getInstance();` 二者的初始化顺序不确定，如果在初始化完成之前调用 `getInstance()` 方法会返回一个未定义的实例。

4 Meyers Singleton

综上所述，在 C++11 以上版本时，用下列方式是最完美的解决方案：

```
class Singleton
{
private:
    Singleton() {}
    ~Singleton() {}
    Singleton(const Singleton &);
    Singleton &operator=(const Singleton &);

public:
    static Singleton &getInstance()
    {
        static Singleton instance;
        return instance;
    }
};
```

5 总结

- Lazy Singleton 通常需要加锁来保证线程安全；
- Eager Singleton 虽然是线程安全的，但存在潜在问题；
- Meyers Singleton 最优雅，但局部静态变量版本在 C++11 后才是线程安全的。

单例模式本质就是统一管理一堆全局变量，用命名空间定义一堆静态方法和静态变量也可以实现。

6 参考文献

1. C++ 单例模式.
2. C++内存屏障（内存顺序）总结.
3. Race condition.
4. Memory model.
5. Memory barrier.